Representação Matemática de Texto Utilizada em PLN

Fabrício Galende Marques de Carvalho

OBS: Esse texto foi produzido com o proposito de servir como um guia de estudo da disciplina Processamento de Linguagem Natural, ministrada por Fabrício Galende Marques de Carvalho, seja em cursos de graduação, pós-graduação ou extensão universitária. É permitida sua livre utilização e reprodução por parte dos alunos matriculados e que tenham concluído cursos dessa área que tenham sido integralmente ministrados pelo autor do material. Qualquer outro uso ou redistribuição deve ser feito somente mediante autorização, por escrito, fornecida pelo autor ao participante. Caso o conteúdo desse trabalho seja usado para subsidiar outras publicações científicas ou tecnológicas, esse deve ser citado da seguinte forma:

Carvalho, F. G. M. Notas de aula do curso Processamento de Linguagem Natural. São José dos Campos, 2025.

Resumo

Esse texto trata da obtenção de diferentes representações matemáticas a partir de conteúdo textual que pode ser utilizado em uma pipeline de processamento de linguagem natural. São abordados modelos baseados em saco de palavras e em embeddings de palavras. Ao longo da discussão são também tratadas medidas de similaridade de representação, que são fundamentais no processo de construção de aplicações, tais como chatbots simples, analisadores de sentimentos e sumarizadores.

1 Introdução

Sistemas computacionais, em essência, só são capazes de efetuar operações elementares sobre números (e.g., somas, subtrações, deslocamentos, etc.), o mesmo vale, portanto, para sistemas de Inteligência Artificial (IA). Tais sistemas, portanto, sempre operam sobre entradas e saídas numéricas que, adicionalmente, necessitam de alguma estruturação/organização.

Conforme foi visto, o conteúdo textual é desestruturado e não numérico. Para que algoritmos de IA possam ser utilizados em tarefas típicas de PLN, tais como análises de sentimentos, classificação, sumarização, busca semântica, etc, é necessário que o texto seja convertido para uma representação numérica compatível com esses algoritmos.

O processo de conversão de conteúdo linguístico textual para uma representação numérica, passível de utilização por algoritmos computacionais numéricos é denominado de Engenharia de Características (do inglês, *Feature Engineering*). Em essência, esse processo recebe como entrada um conjunto de palavras (em geral já pré-processadas), e fornece como saída uma representação numérica. Essa representação

tipicamente é vetorial ou matricial e depende fortemente do corpus que foi utilizado como referência. Esquematicamente, esse processo é ilustrado na Figura 1.



Figura 1: Processo de engenharia de características.

Quando a representação numérica de conteúdo textual passa a ser utilizado para a execução de tarefas tipicamente associadas à linguagem natural tem-se aquilo que é chamado de **modelo de linguagem**. Alguns autores utilizam essa terminologia somente quando o modelo numérico é utilizado para prever sequências de palavras. A previsão, para esses casos, tipicamente é baseada em modelagem probabilística ou baseada em redes neurais artificiais (RNAs). Quando se efetua a consideração probabilística, um modelo de linguagem pode ser esquematizado tal como o processo da Figura 2

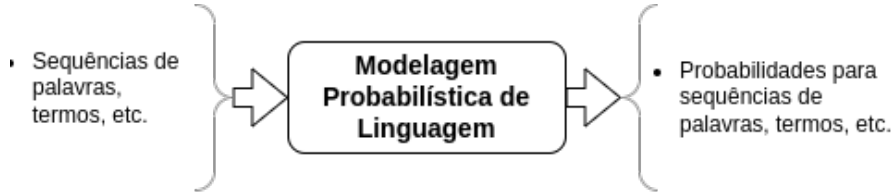


Figura 2: Modelagem probabilística de linguagem.

Neste texto, serão estudados modelos de representação numérica de texto e modelos probabilísticos de linguagem simplificados. Para fins de simplificação, ao longo desse texto, os termos representação e modelo de linguagem serão utilizados como sinônimos.

2 Representação por Saco de Palavras

Como o próprio nome já diz, trata-se de um modelo de representação que considera que um determinado documento (ou sentença, ou frase), pode se representado por um conjunto de palavras, independentemente da ordem na qual essas palavras aparecem no texto. Por esse motivo é chamado de "saco"de palavras (BOW, do inglês, ***Bag Of Words***).

Esquematicamente, cada palavra ou termo representa uma dimensão na representação numérica vetorial do texto considerado. Se considerarmos um corpus de referência contendo N palavras/termos distintos, então o documento a ser convertido para esse modelo de representação bag of words será um vetor do espaço N dimensional. A Figura 3 ilustra esquematicamente, em termos de entradas e saídas, o processo de obtenção da representação matemática do texto a partir de um corpus de referência e de um documento de entrada. Já a Figura 4 ilustra detalhes do algoritmo de extração de características para o modelo de saco de palavras.

Exemplo: Consideremos um corpus de referência fictício dado por:



Figura 3: Modelagem bag of words.

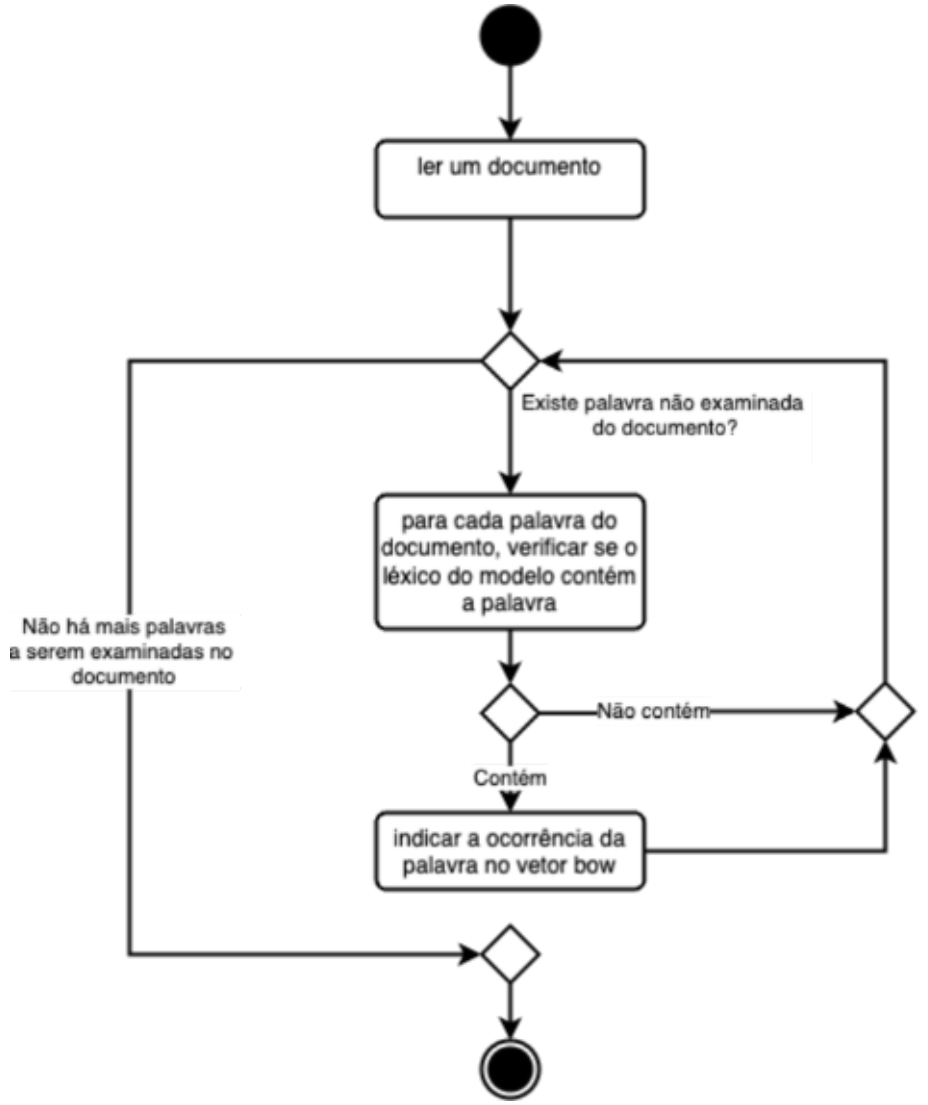


Figura 4: Algoritmo de extração de características para o BOW

"Eu gostei do produto. A qualidade é ótima!"

Para esse corpus, tem-se o seguinte conjunto de palavras distintas: D = { eu, gostei, do, produto, a, qualidade, é, ótima}. Dessa forma, a dimensionalidade do espaço de representação utilizando bag of words será N = 8. Considerando o documento de entrada formado pela frase única s = "Ótima qualidade!", tem-se a seguinte representação bag of words para esta entrada:

$bow(s) = [0, 0, 0, 0, 0, 1, 0, 1]$.

Note-se que comumente uma frase ou documento são representados por um vetor linha, em que cada feature (palavra) corresponde a uma coluna distinta. Já um conjunto N documentos (ou N frases), seriam

representados por uma matrix NxM. Algumas vezes, quando se utiliza a representação com vetores coluna, a matriz do conjunto de documentos seria MxN, sendo cada linha correspondente a uma feature e cada coluna correspondente a um documento.

Ainda com relação a essa representação, quando se considera somente a ocorrência ou não de uma palavra do documento ou frase, diz-se que a representação é do tipo saco de palavras binário (do inglês, *binary bag of words*). Quando se consideram as contagens das palavras no documento ou frase, tem-se a representação de saco de palavras baseada em contagem (do inglês, *count vectorization bag of words.*)

Exemplo: Considerando um documento do tipo "Ótima qualidade é a qualidade do produto!", tem-se que a representação do tipo saco de palavras por contagem, utilizando o mesmo corpus de referência do exemplo anterior, será:

$bow_c(s) = [0, 0, 1, 1, 1, 2, 1, 1]$.

Algumas vantagens da representação por saco de palavras incluem:

- É uma representação extremamente simples de ser implementada.
- É de fácil entendimento.
- Não requer análise gramatical ou processamentos mais complexos.

Como desvantagens desse tipo de representação pode-se citar:

- Pode ocorrer a perda de informação contextual, relação de ordem, etc.
- Sem o adequado pré-processamento, gera entradas de alta cardinalidade.
- Potencial obtenção de vetores esparsos, podendo gerar dificuldade no treinamento do modelo.

3 Representação por Saco de Palavras N-Gram

Trata-se de uma generalização do modelo de saco de palavras mas considerando o vetor de características montado a partir de conjuntos de palavras, preservando a ordem em que aparecem em uma frase ou sentença.

O princípio "por trás do n-gram" está no fato de que palavras que aparecem frequentemente juntas tipicamente se referem a algo com algum significado relevante.

Algumas vantagens desse modelo incluem:

- Por continuar a ser um saco de palavras, preserva simplicidade.
- É de fácil entendimento.
- Retém o significado e a estrutura parcial de uma frase/oração/período considerando termos próximos.

Como desvantagem desse modelo, pode-se citar:

- O número de combinações de palavras existentes nos n-grams pode se tornar elevado, fazendo com que o vetor de características se torne de elevada dimensão.
- Potencial obtenção de vetores esparsos, podendo gerar dificuldade no treinamento do modelo de IA.

4 Transformação TF-IDF

Um problema que pode ocorrer ao se efetuar a representação utilizando-se a contagem de palavras em grandes corpora é o fato de que termos muito frequentes podem acabar dominando o vetor de representação. Para amenizar esse efeito, pode-se efetuar uma transformação matemática baseada não somente na frequência de ocorrência do termo na frase/documento de modo isolado, mas também considerando sua ocorrência nos corpora de referência. Essa transformação é denominada de TF-IDF (do inglês, Term Frequency - Inverse Document Frequency) e é composta por duas métricas. A primeira baseada na frequência do termo no na frase/documento isolado e a segunda baseada na ocorrência do termo no conjunto de documentos do corpora de referência.

Seja $\vec{W}_i = [w_1, w_2, .., w_m]$ o vetor representativo de um determinado documento d_i e sejam os corpora de referência compostos pelo conjunto de documentos $D = \{d_1, d_2, ..., d_N\}$. Então, para uma determinada característica w_{jd_i}, pertencente ao documento d_i a transformação TF-IDF é definida por

$$tfidf(w_{jd_i}) = tf(w_j, d_i).idf(w_j, D) \tag{1}$$

Na Equação (1), $tf(w_j, d_i)$ corresponde à contagem do termo w_j no documento d_i, já $idf(w_j, D)$ é dado por

$$idf(w_j, D) = Log\left(\frac{N}{df(w_j, D)}\right). \tag{2}$$

Na Equação (2), $df(w_j, D)$ é a contagem de documentos em D nos quais w_j aparece, ou seja, a frequência, em termos de documentos, considerando D.

Quando o vetor de características potencialmente inclui termos que não fazem parte do corpora de referência, pode-se utilizar a seguinte modificação da fórmula (1) para se evitar a divisão por zero:

$$tfidf(w_{jd_i}) = [tf(w_j, d_i) + 1].\left[1 + Log\left(\frac{N+1}{1+df(w_j, D)}\right)\right] \tag{3}$$

Considerando o fato de que é mais importante a direção do vetor de características do que sua magnitude, é comum que, após se efetuar a transformação TF-IDF, o vetor seja normalizado utilizando-se norma euclidiana (conhecida como norma L2).

$$tfidf_{norm}(\vec{W}_i) = \frac{tfidf(\vec{W}_i)}{||(\vec{W}_i)||} \tag{4}$$

Exemplo: Considerando o corpus de referência composto pelas frases

Frase 1: "gostei, gostei muito do sapato!"

Frase 2: "Não gostei do tênis."

Tem-se o léxico da representação dado por [do, gostei, muito, não, sapato, tênis]. A partir desse léxico, obtem-se a seguinte representação baseada em contagem BOW:

bow(Frase 1) = [1 2 1 0 1 0]

bow(Frase 2) = [1 1 0 1 0 1]

Efetuando-se o cálculo do TF-IDF com o fator de suavização (+1 para evitar divisões por zero), obtém-se as seguintes representações normalizadas :

$tfidf_{norm}(bow(Frase1)) = [0.334251, 0.668501, 0.469778, 0.0, 0.469778, 0.0]$

$tfidf_{norm}(bow(Frase2)) = [0.409937, 0.409937, 0.0, 0.576152, 0.0, 0.576152]$

5 Medida de Similaridade Entre Representações

Medidas de similaridade são utilizadas para dizer o quanto um determinado documento, pertencente a um corpus, é similar a outro. Essas medidas são utilizadas por algoritmos de agrupamento e classificação e tornam possível a utilização de diferentes técnicas de inteligência artificial em tarefas de PLN.

Em geral, medidas de de similaridade fazem uso de métricas de distância linear ou angular entre os vetores de características que representam um determinado elemento linguístico (e.g., uma palavra, uma frase, etc.).

Um exemplo de medida de similaridade é o obtida efetuando-se o cálculo do cosseno entre os ângulos formados pelos vetores representativos por cada um dos documentos de um corpus.

Sejam dois documentos d_1 e d_2 representados pelos vetores $\vec{v_1}$ e $\vec{v_2}$, respectivamente. Nesse caso, a medida de similaridade do cosseno entre esses dois documentos será dada por:

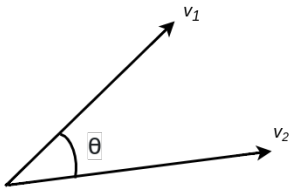


Figura 5: Esquema de medida de similaridade usando cosseno.

$$cos(\theta_{12}) = \frac{\vec{v_1} \cdot \vec{v_2}}{||v_1||||v_2||} \tag{5}$$

Para um corpus de texto contendo M documentos, o resultado do modelo de similaridade 2 a 2 utilizando cosseno será uma matriz simétrica(MxM). Nesse caso, os índices das linhas e das colunas são os responsáveis por indexar os documentos e os valores das células da matriz serão os cossenos (métricas de similaridade).

Uma das principais aplicações dos modelos de similaridade está na utilização de algoritmos de aprendizagem não supervisionada que trabalham através de agrupamento.

Outro exemplo de utilização desse tipo de métrica são os algoritmos de sumarização extrativa. Nesse caso, os documentos que são "mais similares aos demais documentos do corpora"são selecionados como

aqueles representativos do sumário extrativo.

6 Modelagem Utilizando Word2Vec

Os modelos baseados em contagens de palavras que foram vistos anteriormente são ditos modelos tradicionais de características. Esses modelos, apesar de serem fáceis de serem compreendidos, são modelos que tendem a produzir vetores esparsos (contendo muitos zeros) e que não preservam relação de ordem ou similaridade existentes entre palavras em um dado contexto.

Considere-se, por exemplo, duas frases que representam a opinião positiva de dois consumidores, referentes a um certo produto:
Frase 1: Gosto muito do produto!
Frase 2: Amo esse produto!

Efetuando-se a conversão para letras minúsculas e montando-se um léxico de modelo formado pelas palavras 'gosto', 'do', 'produto', 'amo', 'esse', 'produto' tem-se a seguinte modelagem utilizando-se saco de palavras:
$doc_1 =$ ['gosto do produto']
$doc_2 =$ ['amo esse produto']
$bow_1 = [1, 1, 1, 0, 0]$
$bow_2 = [0, 0, 1, 1, 1]$

Como pode ser observado, os bag of words representativos de documentos muito similares são diferentes e incluem 40% de zeros (informação não relevante) no vetor de características. Esse aspecto torna a utilização dessa representação problemática para tarefas tais como classificação e reconhecimento de padrões.

Pensando em resolver o problema de elevada dimensionalidade e de vetores esparsos dos modelos tradicionais, uma equipe do Google propôs, por volta de 2013, um modelo que gera um vetor pouco esparso e denso para cada uma das palavras pertencentes a um corpus.

Esses modelos, conhecidos como "word embeddings" capturam tanto a relação de ordem como o contexto relacionado a uma dada palavra.

Em essência, um modelo de "word embedding"treina uma rede neural, contendo múltiplas camadas de modo que, dada uma palavra de entrada, codificada com *One-Hot*, a rede tenta prever qual o contexto (i.e., palavras ao redor da entrada). Esse esquema é conhecido como skip-gram. A Figura 6 ilustra esquematicamente a modelagem utilizando embeddings e word2vec enquanto que a Figura 7 ilustra o esquema da RNA utilizada no treinamento de acordo com a arquitetura skip-gram.

Caso a camada que processa a entrada da rede seja composta inteiramente por neurônios lineares, as colunas da matriz de entrada armazenarão as codificações (embeddings) das palavras do léxico do modelo. Nesse caso, obter a representação word2vec após o treinamento da RNA passa simplesmente pela recuperação da coluna correspondente à palavra nessa matriz matriz.

Além da questão contextual, a utilização desse tipo de representação permite ao desenvolvedor especificar qual o tamanho do vetor representativo de uma determinada palavra ou frase. Dessa forma

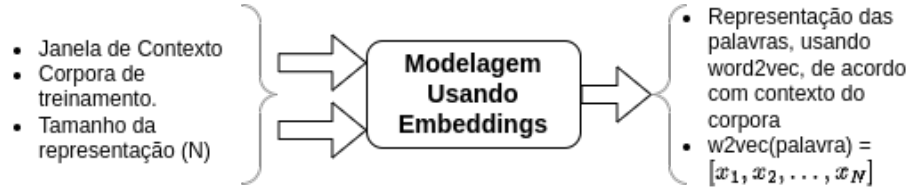


Figura 6: Processo de obtenção de embedding usando word2vec.

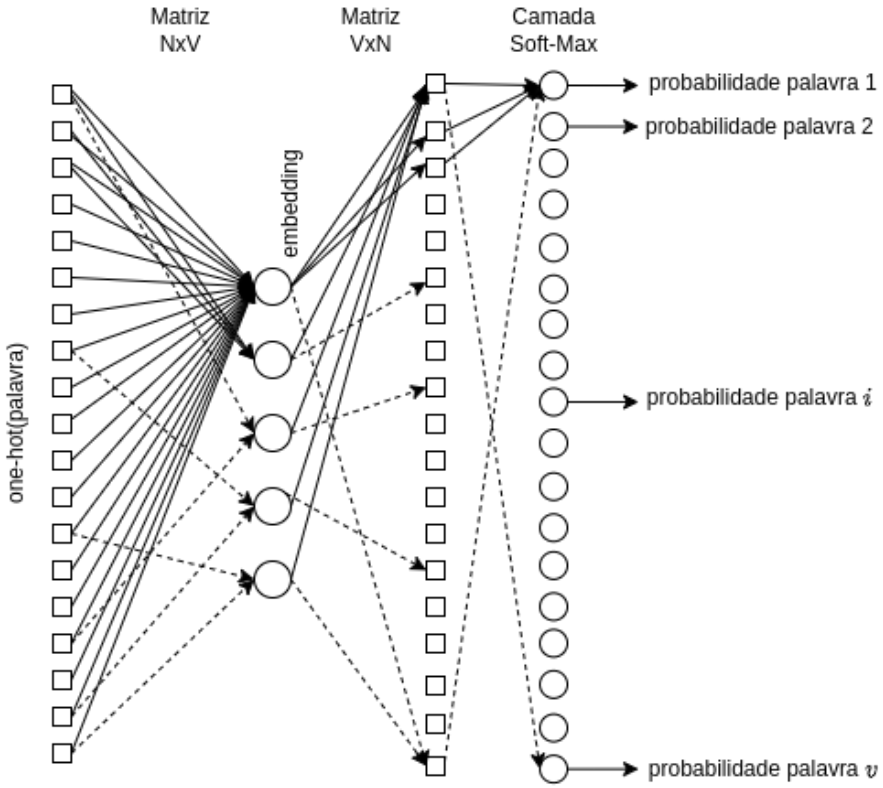


Figura 7: Processo de obtenção de embedding usando word2vec.

os vetores serão não esparsos, ou seja, incluirão menos zeros, e possuirão representações similares para palavras que possuem significados similares em um determinado contexto.

Para o caso de frases, a modelagem utilizando embeddings, obtém o vetor representativo de uma determinada frase considerando a média de seus embeddings representativos de cada uma das palavras. Ou seja, considerando que para uma palavra w_i, o embedding correspondente é o vetor $e_i = [e_{i1}, e_{i2}, ..., e_{iN}]$, então o embedding correspondente à frase $ef = e(\text{frase}) = [ef_1, ef_2, ..., ef_N]$ contendo M palavras é dado por

$$ef_k = \frac{\sum_{i=1}^{M} e_{ik}}{M} \tag{6}$$

7 Exercícios e Problemas

OBSERVAÇÃO: Na solução dos exercícios e problemas seguintes, prepare uma pequena apresentação explorando os conceitos envolvidos no problema/exercício, justificando e explicando em detalhes sua resposta. Além disso, caso o problema requeira o desenvolvimento de um programa computacional e não tenha sido mencionada nenhuma linguagem de programação, utilize a linguagem de programação Python 3.* e forneça como resposta o código-fonte, o procedimento de execução e o arquivo referente às dependências (e.g., requirements.txt, package.json, etc). Caso o enunciado trate de um pequeno exemplo de código executável e não de um sistema, pode ser utilizada a tecnologia do Jupyter Notebook.

TC.1. Qual tipo de problema pode surgir na montagem de um modelo do tipo bag of words caso etapas tais como a remoção de caracteres especiais (ex. sinais de pontuação), stemização, lematização e conversão para minúsculas/maiúsculas não sejam efetuadas? Ilustre isso para as seguintes frases que fazem parte de um mesmo corpus de texto (i.e.: são usadas para a montagem do léxico do modelo):

- Frase 1: Eu quero tomar água!
- Frase 2: eu, prefiro tomar café.
- Frase 3: Eu preferiria tomar café quente.
- Frase 4: Eu queria tomar água?

Na sua resposta, ilustre como seriam representados o léxico do modelo e os modelos de frase do tipo bag of words correspondentes. Na sua resposta inclua também qual a cardinalidade do léxico de palavras e dos modelos com e sem os elementos da pipeline de pré-processamento.

TC.2. Qual a relação entre as etapas de pré-processamento de texto e a redução de dimensionalidade quando se lida com extração de características? Ilustre isso considerando 3 exemplos que façam uso de stemização e/ou lematização. Ilustre, também, como os processos de stemização e lematização podem interferir na perda do significado de uma frase que será manifestado pela obtenção de vetores bag of words idênticos para frases que são semanticamente diferentes.

TC.3. Descreva como ficaria o léxico do modelo e o vetor de características n-gram para n=1, 2 e 3 para os seguintes documentos pertencentes ao mesmo corpus:

- Frase 1: Eu não gostei do produto e o produto parece ruim.
- Frase 2: O produto parece bom.
- Frase 3: O produto parece ruim.
- Frase 4: Parece muito ruim esse produto.

TC.4. Para o exercício TC.3., considerando um modelo bag of words, com n=1, mostre como se calcula o valor da transformação TFIDF para as palavras produto e ruim, ambas na frase 1. Utilize

as expressões básicas que não consideram o problema de divisão por zero e considere logaritmos naturais no processo de cálculo. Adicionalmente, forneça o valor dos vetores bag of words, com transformação TFIDF para todas as frases mostradas no exercício.

TC.5. Considere as seguintes frases utilizadas em um sistema de processamento de linguagem natural:

- Frase 1: Gostaria de pedir ajuda para alguém.
- Frase 2: Gostaria de ajuda.
- Frase 3: Gostaria de sair do sistema.
- Frase 4: Sair do sistema!
- Frase 5: Reiniciar o sistema!

Considerando um modelo do tipo bag of n-words, com n= 1 e n =2, monte o léxico do modelo e o vetor de caraterísticas representativo de cada frase considerando:

- A mera contabilização dos n-gramas na frase.
- A aplicação da transformação TFIDF para o vetor de contabilização.

Diga se n=1 ou n=2 afeta o tamanho e o valor das componentes do vetor de características obtido para o modelo. Na sua resposta, especifique os valores dos termos IDF para os unigramas e bigramas. Apresente o detalhamento do cálculo de pelo menos 4 IDFs, sendo 2 para unigramas e 2 para bigramas.

PC.1. Baseando-se nos exemplos fornecidos pelo professor, que fazem uso da biblioteca scikit-learn, ilustre a obtenção de um vetor do tipo bag of words com transformação do tipo TFIDF para três documentos que representem reviews de produtos em um site de e-commerce. Execute todas as etapas de pré-processamento necessárias para normalizar os dados. Uma vez obtidos os vetores, faça o cálculo de similaridade utilizando a função cosseno e diga quais reviews são mais similares e quais reviews são mais distintas.

PC.2. Baseando-se nos exemplos fornecidos pelo professor, que fazem uso da biblioteca scikit-learn, ilustre a obtenção de um vetor do tipo bag of words com transformação do tipo TFIDF para três documentos que representem solicitações de atendimento que chegam em um service desk. Execute todas as etapas de pré-processamento necessárias para normalizar os dados. Uma vez obtidos os vetores, faça o cálculo de similaridade utilizando a função cosseno e diga quais solicitações são mais similares e quais solicitações são mais distintas. Ilustre a matriz de similaridade.

PC.3. Baseando-se nos exemplos fornecidos pelo professor, que fazem uso da biblioteca scikit-learn, ilustre a obtenção vetores do tipo embeddings para três documentos que representem reviews de produtos em um site de e-commerce. Execute todas as etapas de pré-processamento necessárias para normalizar os dados. Uma vez obtidos os vetores, faça o cálculo de similaridade utilizando a função cosseno e diga quais reviews são mais similares e quais reviews são mais distintas. Ilustre a matriz de similaridade.

PC.4. Baseando-se nos exemplos fornecidos pelo professor, que fazem uso da biblioteca scikit-learn, ilustre a obtenção vetores do tipo embeddings para três documentos que representem olicitações de atendimento que chegam em um service desk. Execute todas as etapas de pré-processamento necessárias para normalizar os dados. Uma vez obtidos os vetores, faça o cálculo de similaridade utilizando a função cosseno e diga quais solicitações são mais similares e quais solicitações são mais distintas. Ilustre a matriz de similaridade.

PC.5. Considere o seguinte problema: Desenvolvimento de um motor de busca semântica utilizando repesentação do tipo Bag of Words e métrica de similaridade de cosseno. O cenário consiste em uma interface de busca textual na qual o usuário pode efetuar uma busca em um banco de dados de imagens de sensoriamento remoto. Cada imagem é catalogada de acordo com os seguintes metadados:

- Categoria de imagem (vegetação, solo exposto ou água).
- Data da imagem (formato dd/mm/yyy).
- Origem da imagem (e.g.: satélite, drone, etc.).
- Arquivo da imagem em si (blob ou caminho para o disco para encontrar a imagem.)

O motor de busca deve funcionar da seguinte forma:

- O usuário informa textualmente uma consulta simples desejada (e.g., Quantas imagens estão catalogadas?).
- O motor, que faz uso de PLN, pré-processa o texto de entrada e gera uma lista de tokens normalizados.
- Essa lista de tokens é convertida para uma representação do tipo BOW.
- A representação BOW é comparada, em termos de similaridade de cosseno, com uma série de BOWs de referência (i.e, consultas para as quais se sabe qual deve ser a resposta e qual a consulta SQL correspondente.)
- De acordo com a representação BOW de referência que possui maior similaridade com a entrada do usuário, monta-se a consulta SQL correspondente e retorna-se com a resposta para o usuário.

Exemplo: Perguntas do tipo “Quantas imagens estão catalogadas?” ou “Há quantas imagens?”, após passarem pelo motor de busca devem ser mapeadas para uma SQL query do tipo

```
select count(*) from metadata_table;
```

E devem retornar algo como “No total, há X imagens cadastradas”. Caso o usuário queira, por exemplo, exibir todas as imagens de vegetação, uma consulta do tipo “Quais imagens possuem vegetação?” deve ser mapeada para a query que retorne a lista das imagens e que as exiba na interface.

Sua aplicação deve ser de página única, deve utilizar Python, Framework Flask, PostgreSQL e fazer uso das componentes de pipeline de PLN estudadas em sala, com representação BOW. Não se esqueça de tornar sua aplicação robusta a erros de digitação.

PC.6. Repita o problema PC.5., mas substituindo, no stack de tecnologias, o PostGreSQL pela biblioteca Pandas. Para esse caso, sua base de metadados deve ser armazenada em um arquivo .csv e suas consultas devem ser efetuadas sobre o dataframe correspondente. Na coluna que armazena o arquivo de imagem propriamente dita, pode ser utilizado o caminho absoluto ou relativo para o arquivo de imagem, de modo que permita sua busca e carregamento na interface.

PC.7. Repita o problema PC.5. mas fazendo uso de imagens de pessoas. Seu banco de imagens deve conter fotos de pessoas diversificadas e com os seguintes metadados:

(a) Nome da pessoa;

(b) Etnia da pessoa;

(c) Cor do cabelo da pessoa;

(d) Faixa etária da pessoa (e.g., 0-5, 5-12, etc.).

Seu sistema de busca deve permitir a exibição de resultados que selecionem categorias de imagens de acordo com os metadados e, assim como nos problemas anteriores, exibir as fotos e demais dados das pessoas que satisfazem os critérios definidos.

PC.8. Utilizando a representação *bag of words* com transformação TF-IDF, desenvolva um chatbot que responda algumas questões frequentemente perguntadas em um contexto de uma escola (e.g: FAQ: Onde encontro o horário das disciplinas? Qual o horário das aulas? etc.). Defina claramente o contexto do seu chatbot (i.e., quais categorias de perguntas podem ser respondidas) e utilize a métrica de cosseno e votação por maioria para definir a resposta do seu chat. Desenvolva uma interface web simples e efetue todas as etapas de pré-processamento aplicáveis ao seu sistema. Utilize persistência de dados e não se esqueça de utilizar as etapas de pré-processamento estudadas ao longo do curso.

PC.9. Utilizando a representação *bag of words* com transformação TF-IDF e todas as etapas de pré-processamento, desenvolva um sistema em que um reporter possa fazer o cadastramento de um texto jornalístico e, após cadastrar o texto, o sistema o classifica e o publica em uma interface de notícias do jornal. A interface de notícias deve possuir ao menos 5 categorias (e.g.: esporte, educação, saúde, etc.). Na interface de cadastro o repórter entra com o texto e com uma categoria sugerida. Ao ser processada, a notícia é classificada automaticamente dentro de um dos temas utilizados no treinamento do sistema. Caso o tema divirja daquele informado pelo repórter, o texto precisa ir para um estado que depende de um revisor antes de ser publicado. Caso o tema coincida, o tema é automaticamente publicado, na interface temática do jornal, e é posicionado de acordo com a ordem decrescente de data (i.e., da notícias mais nova para a mais antiga, na mesma interface.).

PC.10. Utilizando a modelagem *bag of words* com transformação TF-IDF, desenvolva um sistema que, a partir de um texto relacionado a um determinado assunto de disciplina básica (e.g.: biologia, química, matemática, literatura, geografia, etc.) automaticamente o classifica e o publica em uma determinada interface web relacionada ao tema. Seu sistema deve aceitar como entrada um documento HTML obtido a partir de uma raspagem web ou então fornecido diretamente pelo usuário (as duas formas devem ser implementadas). Como saída, o sistema identifica a disciplina e a publica no sistema de aprendizagem com classificação automática. Utilize todas as etapas de pré-processamento aplicáveis e teste o seu sistema em páginas web com conteúdos de disciplina (obs: para teste, faça a raspagem de uma página inserida no seu próprio github pages, por exemplo).

PC.11. Efetuando raspagem de dados a partir de uma página de internet que contenha avaliações de produtos, desenvolva um analisador de sentimentos que classifique uma opinião como negativa, positiva ou neutra. Uma opinião negativa, gera um ponto, uma opinião positiva, gera cinco pontos, já uma opinião neutra gera três pontos. Após avaliar uma série de opiniões o sistema efetua a publicação das mesmas opiniões classificadas e gera uma classificação (com estrelas), correspondente à nota média das avaliações. Utilize a modelagem TF-IDF aplicada à representação BOW e, adicionalmente, efetue uma classificação por votação ou utilizando a biblioteca scikit-learn, KNeighborsClassifier com feature vector dado pela representação BOW com TF-IDF.

PC.12. Desenvolva um sumarizador extrativo que funciona da seguinte forma: 1. O sistema permite que o usuário entre com um texto (e.g.: arquivo .txt via upload ou digitando o texto diretamente em um campo de formulário); 2. Para cada frase, o sistema calcula sua representação BOW, com TF-IDF, e sua similaridade com as demais frases do texto. 3. Para cada frase, o sistema calcula o somatório da similaridade dessas com as demais. 4. As frases que contém a maior similaridade com o restante do texto, são ordenadas do maior somatório para o menor somatório. As N (parâmetro configurável) primeiras frases constituem o sumário extrativo do texto original. Além do sistema, apresente exemplos de texto e de sumários extrativos gerados a partir dessa estratégia. Como sugestão, utilize reviews de produtos, notícias, etc.

PC.13. Desenvolva uma aplicação que funcione como um filtro de spam. Seu sistema deve receber como entrada um arquivo de texto ou um texto digitado diretamente pelo usuário em um campo/formulário (as duas opções devem ser possíveis). A saída deve ser sua classificação com spam ou não spam, com a devida probabilidade do classificador. Utilize como features de entrada o vetor BOW com transformação TF-IDF obtido a partir da entrada do usuário. Treine seu modelo com alguns exemplos (crie ao menos 10 exemplos positivos e 10 exemplos negativos). Utilize como classificador o modelo de regressão logística provido pela biblioteca scikit-learn.

Referências